

Name: V Adarsh Reddy

Reg No: 192525412

Course: CSA07 - Computer Networks

Assessment Tool 2 - Pseudocode Writing Task (Annexure A)

Question 1: Throughput and Packet Loss Calculation

Problem: Calculate network throughput (bps) and packet loss (%) given data received, time, packets sent, and packets received. Validate inputs, classify network as Efficient/Inefficient, and display a summary.

Pseudocode:

BEGIN

 // Step 1: Input values

 INPUT total_data_bits // e.g. 8000 bits

 INPUT total_time_seconds // e.g. 4 seconds

 INPUT packets_sent // e.g. 200

 INPUT packets_received // e.g. 180

 // Step 2: Validation to avoid division-by-zero errors

 IF total_time_seconds <= 0 THEN

 DISPLAY "Error: Time must be greater than zero."

 STOP

 ENDIF

 IF packets_sent <= 0 THEN

 DISPLAY "Error: Packets sent must be greater than zero."

 STOP

 ENDIF

 // Step 3: Calculate throughput (bits per second)

 throughput = total_data_bits / total_time_seconds

 // Step 4: Calculate packet loss percentage

 packets_lost = packets_sent - packets_received

 packet_loss_percentage = (packets_lost / packets_sent) * 100

 // Step 5: Classify network performance

 IF throughput > 1500 AND packet_loss_percentage < 10 THEN

 network_status = "Efficient"

 ELSE

 network_status = "Inefficient"

ENDIF

// Step 6: Display complete summary

DISPLAY "—— Network Performance Summary ——"

DISPLAY "Throughput: " + throughput + " bps"

DISPLAY "Packet Loss: " + packet_loss_percentage + " %"

DISPLAY "Network Status: " + network_status

DISPLAY "—————"

END

Explanation:

Throughput = 8000 bits / 4 s = 2000 bps. Packet loss = $(200-180)/200 * 100 = 10\%$. Since throughput (2000) > 1500 but packet loss (10%) is not < 10%, the network is classified as Inefficient. The validation checks prevent divide-by-zero runtime errors when time or packets sent are zero.

Question 2: Bandwidth Monitoring and Overload Detection

Problem: Continuously monitor bandwidth utilization of multiple network links, store readings in an array/list, and recommend switching to a backup link when utilization exceeds 80%.

Pseudocode:

BEGIN

DECLARE link_names[] // list of link identifiers e.g. Link1, Link2, ...

DECLARE bandwidth_usage[] // stores utilization % for each link

DECLARE THRESHOLD = 80 // overload threshold in percentage

DECLARE monitoring = TRUE

INITIALIZE link_names WITH branch link identifiers

WHILE monitoring == TRUE DO

 FOR EACH link IN link_names DO

 usage = GET_CURRENT_BANDWIDTH_USAGE(link) // sensor/simulation reading

 STORE usage INTO bandwidth_usage[link]

 DISPLAY "Link: " + link + " | Usage: " + usage + "%"

 IF usage > THRESHOLD THEN

 DISPLAY "ALERT: " + link + " is overloaded (" + usage + "%)"

 DISPLAY "Recommendation: Switch traffic of " + link + " to backup link"

 SWITCH_TRAFFIC_TO_BACKUP(link)

 ELSE

 DISPLAY link + " is operating normally"

```
ENDIF
ENDFOR
```

```
    WAIT(monitoring_interval) //pause before next monitoring cycle
ENDWHILE
END
```

Explanation:

This algorithm supports efficient bandwidth management by continuously tracking utilization for every link and storing historical readings for analysis. The loop enables real-time, ongoing monitoring rather than a single check. By comparing usage against an 80% threshold, the algorithm proactively detects overloaded links before they cause packet loss or delay, and automatically recommends rerouting traffic to a backup link. This prevents network congestion, balances load across available links, and improves overall reliability of the branch network.

Question 3: Best Path Selection Using Link Quality Score

Problem: For six branch offices, continuously monitor bandwidth, delay, and utilization of each link, compute a link quality score, select the best path, and automatically switch to an alternate path with an alert when a link is overloaded or fails.

Pseudocode:

```
BEGIN
  DECLARE branches[] = [Branch1, Branch2, Branch3, Branch4, Branch5, Branch6]
  DECLARE bandwidth[], delay[], utilization[] // arrays indexed by branch/link
  DECLARE link_score[]
  DECLARE current_path[] // active path per branch
  DECLARE OVERLOAD_LIMIT = 80
  DECLARE monitoring = TRUE

  WHILE monitoring == TRUE DO
    FOR EACH branch IN branches DO
      bandwidth[branch] = GET_BANDWIDTH(branch)
      delay[branch] = GET_DELAY(branch)
      utilization[branch] = GET_UTILIZATION(branch)

      // Step 1: Check for link failure
      IF bandwidth[branch] == 0 THEN
        DISPLAY "ALERT: Link to " + branch + " has FAILED"
        current_path[branch] = SELECT_ALTERNATE_PATH(branch)
        NOTIFY_ADMIN("Link failure on " + branch + ". Switched to alternate path.")
        CONTINUE
      ENDIF
    
```

```

// Step 2: Calculate link quality score
// Higher bandwidth is good, higher delay/utilization is bad
link_score[branch] = (bandwidth[branch] * W1) - (delay[branch] * W2) - (utilization[branch] * W3)

DISPLAY branch + " | Score: " + link_score[branch] + " | Utilization: " + utilization[branch] + "%"

// Step 3: Check for overload
IF utilization[branch] > OVERLOAD_LIMIT THEN
    DISPLAY "ALERT: Link to " + branch + " is OVERLOADED"
    current_path[branch] = SELECT_ALTERNATE_PATH(branch)
    NOTIFY_ADMIN("Overload on " + branch + ". Switched to alternate path.")
ELSE
    // Step 4: Select path with highest score among available paths
    best_path = FIND_MAX_SCORE_PATH(branch)
    current_path[branch] = best_path
    DISPLAY "Best path for " + branch + ": " + best_path
ENDIF
ENDFOR

WAIT(monitored_interval)
ENDWHILE
END

```

Explanation:

This algorithm improves network reliability by continuously re-evaluating each link using a weighted quality score that combines bandwidth, delay, and utilization, rather than relying on static distance-based routing. Routing efficiency is improved since traffic is always guided along the path with the best overall score at that moment. Fault tolerance is achieved through automatic detection of failed or overloaded links, immediate rerouting to alternate paths, and administrator alerts, ensuring communication between headquarters and branches continues with minimal disruption even as network conditions change throughout the day.